

Structures

- The structure in C is a user-defined data type that can be used to group items of possibly different types into a single type.
- The **struct keyword** is used to define the structure in the C programming language.
- The items in the structure are called its **member** and they can be of any valid data type.

C Structure Declaration

- We have to declare structure in C before using it in our program. In structure declaration, we specify its member variables along with their datatype.
- We can use the struct keyword to declare the structure in C using the following syntax:

Syntax

```
struct structure_name {  
    data_type member_name1;    data_type member_name1;  
    ....  
    ....  
};
```

- The above syntax is also called a structure template or structure prototype and no memory is allocated to the structure in the declaration.

Structure Definition

- To use structure in our program, we have to define its instance. We can do that by creating variables of the structure type.
- We can define structure variables using two methods:

◆ Structure Variable Declaration with Structure Template

```

struct structure_name {
    data_type member_name1;
    data_type member_name1;
    ....
    ....
}variable1, variable2, ...;

```

◆ Structure Variable Declaration after Structure Template

```

// structure declared beforehandstruct structure_name variable1,
variable2, .....;

```

Access Structure Members

- We can access structure members by using the (.) dot operator.

Syntax

```

structure_name.member1;
structure_name.member2;

```

In the case where we have a pointer to the structure, we can also use the arrow operator to access the members.

Initialize Structure Members

- Structure members **cannot be** initialized with the declaration. For example, the following C program fails in the compilation.
- We can initialize structure members in 3 ways which are as follows:
 - Using Assignment Operator.
 - Using Initializer List.
 - Using Designated Initializer List.

1. Initialization using Assignment Operator

```

struct structure_name str;
str.member1 = value1;
str.member2 = value2;
str.member3 = value3;

```

2. Initialization using Initializer List

```

struct structure_name str = { value1, value2, value3 };

```

In this type of initialization, the values are assigned in sequential order as they are declared in the structure template.

3. Initialization using Designated Initializer List

```

struct structure_name str = { .member1 = value1, .member2 = value2,
.member3 = value3 };

```

Example:

```

#include <stdio.h>

// Define a structure named 'Person'
struct Person {
    char name[50];
    int age;
};

```

```

int main() {
    // Declare a variable of type 'struct Person'
    struct Person person1;

    // Initialize the values of the structure members
    strcpy(person1.name, "John");
    person1.age = 25;

    // Display the information using the structure
    printf("Name: %s\n", person1.name);
    printf("Age: %d\n", person1.age);

    return 0;
}

```

Output:

```

Name: John
Age: 25

```

typedef for Structures

- The typedef keyword is used to define an alias for the already existing datatype.
- In structures, we have to use the struct keyword along with the structure name to define the variables.
- Sometimes, this increases the length and complexity of the code. We can use the typedef to define some new shorter name for the structure.

Example:

```

// C Program to illustrate the use of typedef with
// structures
#include <stdio.h>

// defining structure
struct str1 {
    int a;
};

```

```

// defining new name for str1
typedef struct str1 str1;

// another way of using typedef with structures
typedef struct str2 {
    int x;
} str2;

int main()
{
    // creating structure variables using new names
    str1 var1 = { 20 };
    str2 var2 = { 314 };

    printf("var1.a = %d\n", var1.a);
    printf("var2.x = %d", var2.x);

    return 0;
}

```

Output:

```

var1.a = 20
var2.x = 314

```

Nested Structures

- C language allows us to insert one structure into another as a member. This process is called nesting and such structures are called nested structures.
- There are two ways in which we can nest one structure into another:

1. Embedded Structure Nesting

In this method, the structure being nested is also declared inside the parent structure.

2. Separate Structure Nesting

In this method, two structures are declared separately and then the member structure is nested inside the parent structure.

Structure Pointer in C

We can define a pointer that points to the structure like any other variable. Such pointers are generally called Structure Pointers. We can access the members of the structure pointed by the structure pointer using the (->) arrow operator.

Example:

```
#include <stdio.h>

struct Student {
    int rollNumber;
    char name[50];
    float marks;
};

int main() {
    // Declare a structure variable
    struct Student student1;

    // Initialize structure fields using dot operator
    student1.rollNumber = 101;
    strcpy(student1.name, "John");
    student1.marks = 85.5;

    // Declare a structure pointer and assign address of student1 to it
    struct Student *ptr = &student1;

    // Access structure fields using the structure pointer and arrow operator
    printf("Roll Number: %d\n", ptr->rollNumber);
    printf("Name: %s\n", ptr->name);
    printf("Marks: %.2f\n", ptr->marks);
    return 0;
}
```

```
}
```

Output:

Roll Number: 101

Name: John

Marks: 85.50

Uses of Structure in C

C structures are used for the following:

- The structure can be used to define the custom data types that can be used to create some complex data types such as dates, time, complex numbers, etc. which are not present in the language.
- It can also be used in data organization where a large amount of data can be stored in different fields.
- Structures are used to create data structures such as trees, linked lists, etc.
- They can also be used for returning multiple values from a function.

Limitations of C Structures

In C language, structures provide a method for packing together data of different types. A Structure is a helpful tool to handle a group of logically related data items. However, C structures also have some limitations.

- **Higher Memory Consumption:** It is due to structure padding.
- **No Data Hiding:** C Structures do not permit data hiding. Structure members can be accessed by any function, anywhere in the scope of the structure.
- **Functions inside Structure:** C structures do not permit functions inside the structure so we cannot provide the associated functions.
- **Static Members:** C Structure cannot have static members inside its body.
- **Construction creation in Structure:** Structures in C cannot have a constructor inside Structures.